Proyecto Final: Plataforma SaaS Multi-empresa (Temática Libre)

Objetivo del Proyecto:

Diseñar, desarrollar, contenerizar y desplegar una plataforma web transaccional bajo el modelo SaaS (Software as a Service) **Multi-empresa**, cuya temática de negocio será definida por cada equipo. El sistema debe respetar Clean Architecture, Vertical Slicing, y ser desplegado en un ecosistema Cloud de costo cero.

Fase 1: Ideación, Análisis y Gobierno Ágil (15%)

El estudiante toma propiedad del producto definiendo un sistema B2B (Business-to-Business).

- **Definición del SaaS:** Pitch de 1 página definiendo la plataforma (ej. *SaaS para Gestión de Barberías*, *SaaS de Inventario para Ferreterías*, *SaaS para Clínicas Dentales*).
- **Diseño Multi-empresa (Multi-tenant):** Modelado del Diagrama Entidad-Relación (ERD) en 3ra Forma Normal (3FN). Todas las tablas transaccionales principales deben incluir su columna `EmpresaId` como llave foránea.
- **Tablero Kanban:** Uso de **GitHub Projects** con Historias de Usuario claras.
- **Contratos de Interfaz:** Definición del diseño de la API (Endpoints, DTOs de entrada y salida).

Fase 2: Construcción del Backend Transaccional (.NET + PostgreSQL) (25%)

El motor del sistema, riguroso, multi-cliente y protegido.

- **Clean Architecture:** Implementación estricta de capas (`Domain`, `Application`, `Infrastructure`, `API`).
- **Persistencia y Aislamiento de Datos:** Entity Framework Core (Code-First). Implementación de **Global Query Filters** (`HasQueryFilter`) para garantizar que las consultas filtren automáticamente por el `EmpresaId` del usuario actual.
- **Seguridad IAM (JWT Claims):** Autenticación con **Tokens JWT**. El `EmpresaId` debe viajar encriptado como un *Claim* dentro del token para que el Backend sepa a qué empresa pertenece el usuario que hace la petición.
- **Validación Semántica:** Artefactos de validación con *FluentValidation* propagando los errores hacia la capa API mediante un Middleware global.

Fase 3: Frontend Web Reactivo (React + TypeScript) (25%)

La interfaz modular y resiliente para los usuarios finales de cada empresa.

- **Arquitectura Vertical Slicing:** Agrupación por funcionalidades (ej. features/auth, features/clientes, features/facturacion).
- **Rutas Protegidas:** Guardias de navegación con react-router-dom para bloquear usuarios sin JWT.
- **Validación Sintáctica en Vivo: Zod + React Hook Form** (mode: 'onChange'), en perfecta simetría con *FluentValidation* del Backend.
- **Resiliencia HTTP:** Extractor universal de errores (el programado en la Clase 16) e Interceptores de Axios para adjuntar el JWT (Bearer) en cada petición.

Fase 4: Contenerización (Dockerización Local) (15%)

Estandarizando el entorno de desarrollo para erradicar el "en mi máquina sí funciona".

- **Dockerfiles:** Dockerfile Multi-stage para la API en .NET (SDK para compilar, Runtime ligero de Alpine para ejecutar).
- **Docker Compose Local:** Creación de un docker-compose.yml que levante la API (.NET) conectada a un contenedor efímero de PostgreSQL para desarrollo local unificado en el equipo.

Fase 5: Despliegue Cloud Automatizado (El *Zero-Cost Stack*) (20%)

Despliegue a producción utilizando infraestructura moderna, Serverless y 100% gratuita para educación.

1. **Base de Datos (Neon.tech o Supabase):** Plataforma de PostgreSQL en la nube, gratuita y sin tarjeta de crédito.
2. **Backend Web Service (Render.com o Fly.io):** Plataforma PaaS conectada a GitHub. Compila el Dockerfile de .NET y expone la API en un dominio HTTPS automático.
3. **Frontend SPA (Vercel o Netlify):** Despliegue del cliente React/Vite en segundos con un CDN global, conectado a la rama main de GitHub.
4. **Integración Continua Básica:** Cada git push a la rama principal actualizará automáticamente todo el ecosistema SaaS en producción.

📋 Rúbrica de Evaluación del Proyecto Final

Criterio de Ingeniería	Deficiente (0-50%)	Aceptable (51-80%)	Excelente (81-100%)
I. Análisis y BD (Multi-empresa)	Lógica de negocio pobre o confusa. Tablas sin aislar por empresa.	Temática definida. ERD normalizado. Manejo funcional de entidades.	Regla Multi-empresa clara (EmpresaId). Restricciones e índices correctos en el ERD. Backlog ágil.

Criterio de Ingeniería	Deficiente (0-50%)	Aceptable (51-80%)	Excelente (81-100%)
II. Backend (.NET Core)	Mezcla de capas. Contraseñas planas. Aislamiento manual o fallido.	Clean Architecture respetada. JWT funcional. EF Core funcionando.	Filtros Globales de EF aplicados. Simetría DTO. Middleware de excepciones interceptando FluentValidation.
III. Frontend (React/Vite)	*God components*. Validaciones con if/else manuales.	Módulos separados. Protección de rutas básicas. Peticiones con Axios.	Arquitectura Vertical. Zod en tiempo real + Extractor de errores HTTP. UX fluida.
IV. DevOps Local (Docker)	No compila en Docker.	Dockerfile básico funcional.	docker-compose.yml levanta el sistema completo en local con un solo comando.
V. Despliegue Cloud	El proyecto solo funciona en localhost.	Despliegue parcial (solo Frontend o solo Backend).	**Full-Stack en la nube:** Vercel (Front) consumiendo Render (Back) conectado a Neon (BD). Todo HTTPS.